YILDIZ TEKNİK ÜNİVERSİTESİ ELEKTRİK-ELEKTRONİK FAKÜLTESİ BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

2025/1 EĞİTİM ÖĞRETİM YILI NESNEYE YÖNELİK PROGRAMLAMA DERSİ PROJE RAPORU

HAZIRLAYANLAR:

Adı Soyadı: Nehir ERVAN

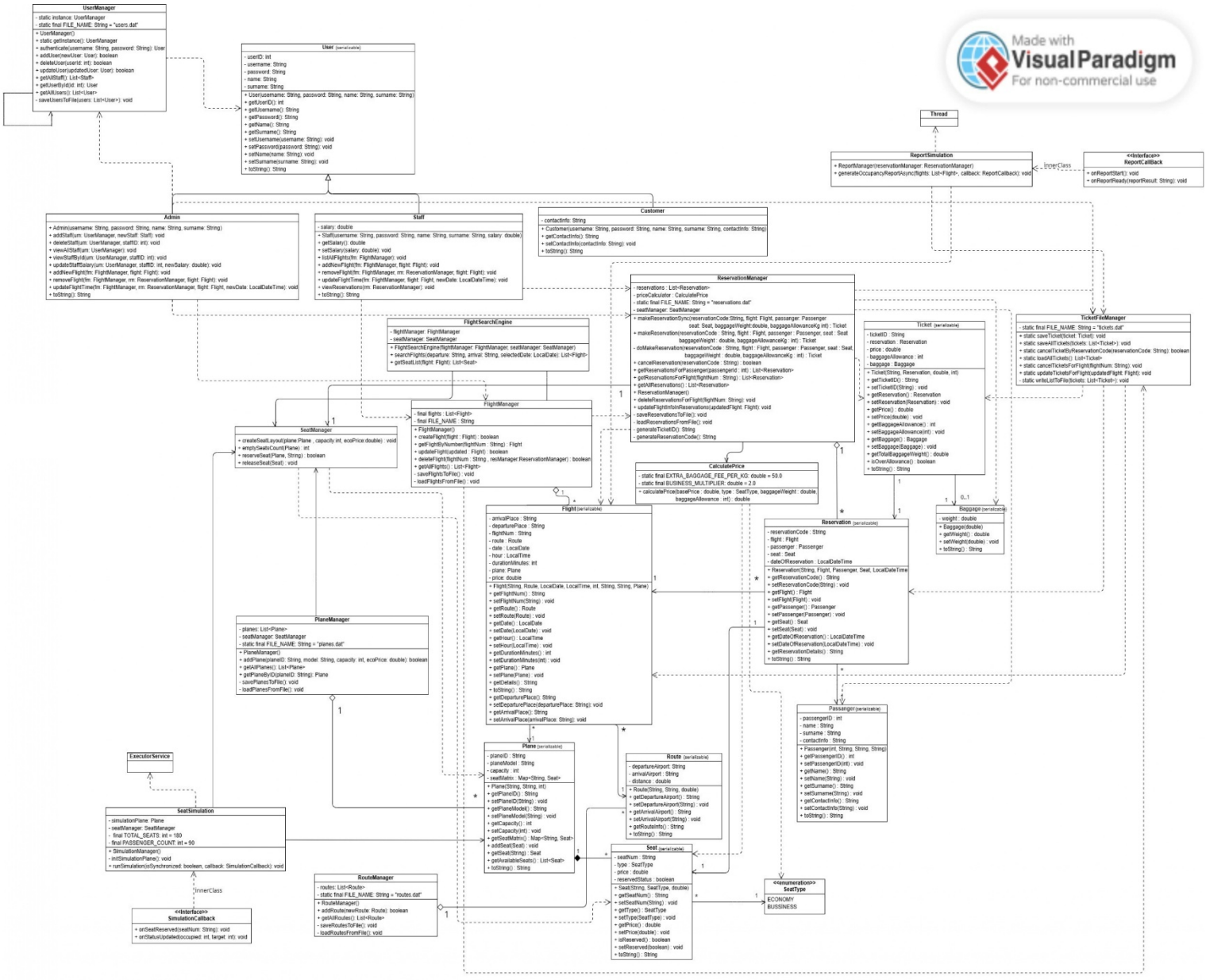
Öğrenci Numarası: 23011052

Öğretim Üyesi: Furkan Çakmak

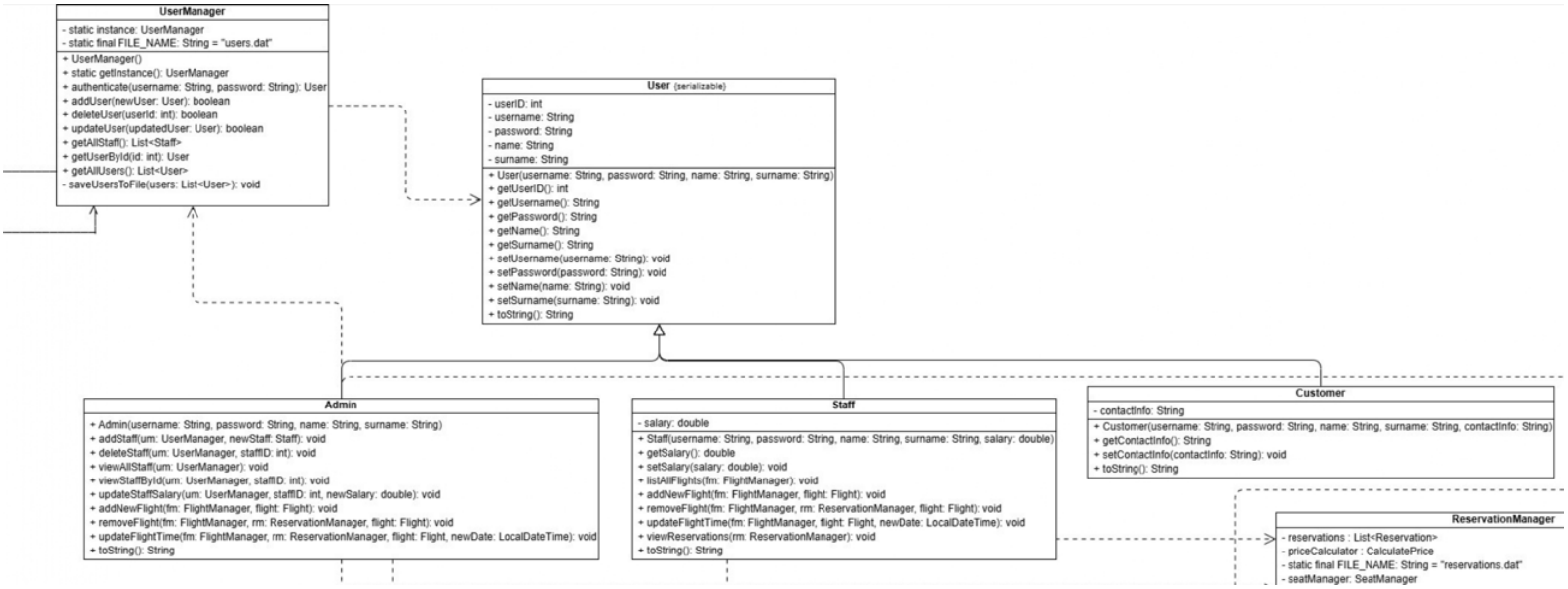
Adı Soyadı: Onur Akyar

Öğrenci Numarası: 23011048

Öğretim Üyesi: Furkan Çakmak


UserManager
User
Admin
Staff
Customer
Thread
ReportSimulation
<<Interface>>
ReportCallBack
ReservationManager
FlightSearchEngine
FlightManager
SeatManager
CalculatePrice
TicketFileManager
Ticket
Flight
Reservation
Baggage
PlaneManager
Plane
Route
Passanger
Seat
RouteManager
ExecutorService
SeatSimulation
<<Interface>>
SimulationCallback
<<enumeration>>
SeatType
ECONOMY
BUSSINESS
InnerClass
Made with
Visual Paradigm
For non-commercial use

UML DİAGRAM



-User Class

*The User abstract class is the Parent class for Admin, Staff, and Customer classes. These classes are derived from the User class (Inheritance).

*It implements the Serializable interface for file writing operations.

Methods:

User(...): Initializes the object by generating a random ID (between 1000-99999) within the Constructor.

Getter/Setter: Manages username, password, name, and surname information.

-Staff Class

*The Staff class is derived from the User class. It has a Dependency relationship with FlightManager and ReservationManager for operational tasks.

Methods:

listAllFlights(FlightManager fm): Lists the current flights in the system.

addNewFlight(...): Adds a new flight with information provided by the user.

removeFlight(FlightManager fm, ReservationManager rm, Flight flight): Deletes the flight and triggers the reservation manager to ensure related tickets/reservations are also deleted.

viewReservations(ReservationManager rm): Displays all reservation records in the system.

-Admin Class

*The Admin class is derived from the User class (Inheritance). It has a Dependency relationship with UserManager for staff operations, FlightManager for flight management, and ReservationManager for synchronization.

Methods:

addStaff(UserManager um, Staff newStaff): Adds new staff to the system via UserManager.

deleteStaff(...): Deletes staff from the system based on ID information via UserManager.

updateStaffSalary(...): Updates staff in the system based on ID information via UserManager.

viewAllStaff(UserManager um): Lists all registered staff (Staff) in the system.

addNewFlight(...): Adds a new flight with information provided by the user.

removeFlight(FlightManager fm, ReservationManager rm, Flight flight): Deletes the flight and triggers the reservation manager to ensure related tickets/reservations are also deleted.

updateFlightTime(...): Updates the flight time and calls TicketFileManager and ReservationManager to synchronize the time on all tickets.

-Customer Class

The Customer class is derived from the User class (Inheritance). It holds contact information (contactInfo) in addition to standard user information.

Methods:

Getter/setter: Modifies the customer's phone number or allows it to be called from another class.

-User Manager Class

*The UserManager class has a Dependency relationship with the User class as it manages the list of User objects within a method.

*It is created using the Singleton design pattern. (Only one instance runs throughout the application).

Methods:

getInstance(): Returns the single instance of the class.

authenticate(String username, String password): Matches the logging-in user's information with users in the list. Returns the User object if there is a match.

addUser(User newUser): Adds a new user based on the provided information.

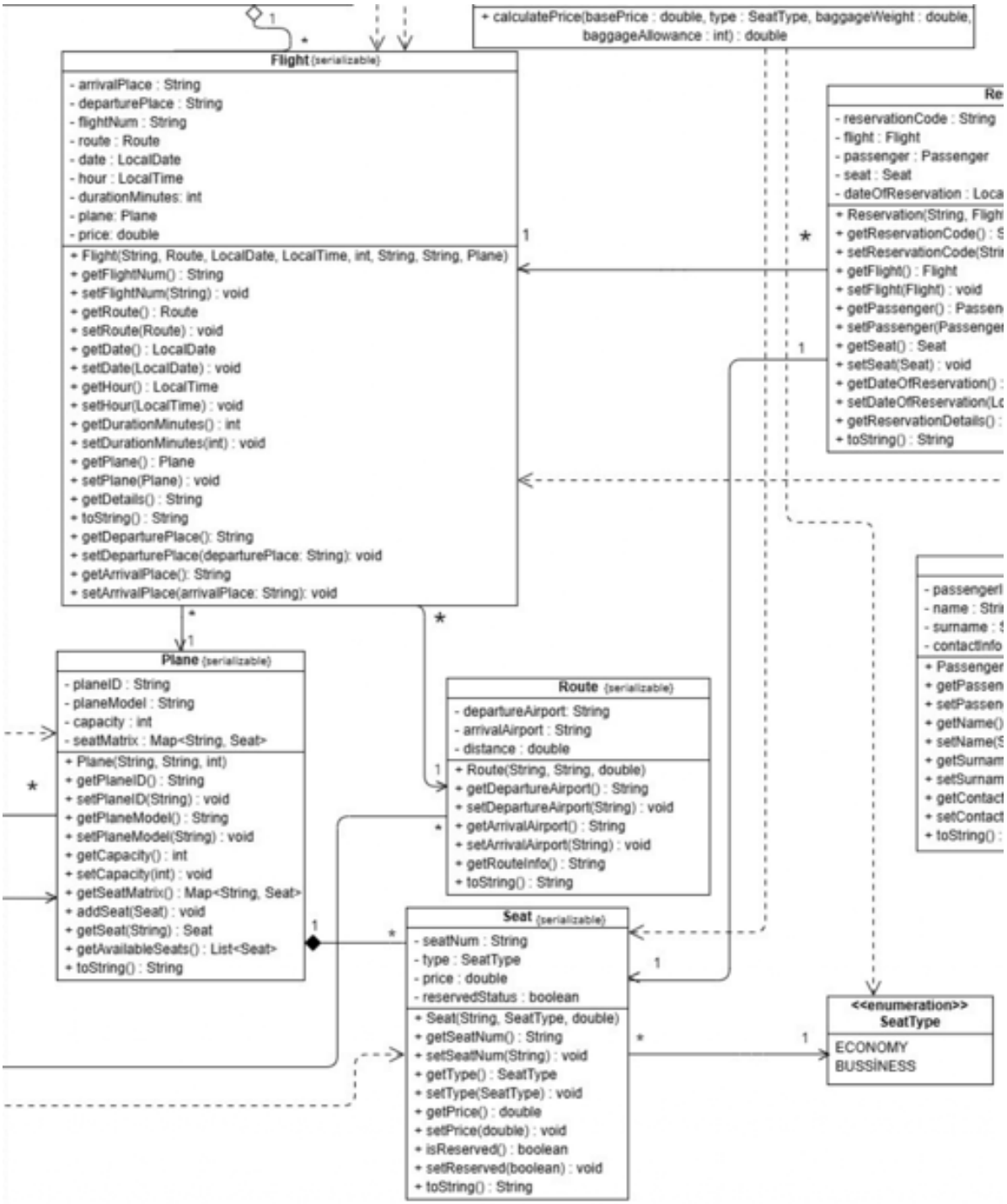
deleteUser(int userId): Deletes the user from the list based on the ID.

getAllStaff(): Filters the user list and returns only those who are Staff.

saveUsersToFile(): Saves the user list to the users.dat file.

getAllUsers(): Reads the user list from the file and loads it into memory.

Flight Management Module



-Flight Class

*The Flight class has an association relationship with the Route class. The Flight class contains a route object and performs route operations through it.

*The Flight class has an association relationship with the Plane class. The Flight class contains a plane object.

Methods:

Set and get methods: Used to perform modification and retrieval operations.

-Plane Class

*The Plane class has an association (composition) relationship with the Seat class. The Plane class contains a structure (Map) named seatMatrix, which holds Seat objects. The plane object manages multiple seat objects within it.

Methods:

Set and Get Methods: Used to modify and retrieve properties such as the plane's ID, model, and capacity. The set methods include checks to prevent invalid inputs like empty values or negative capacity.

addSeat(Seat seat): Adds a new seat to the plane. When adding, it checks if the capacity is full and if the same seat number has already been added.

getAvailableSeats(): Scans all seats on the plane and returns a list of unreserved (empty) seats.

isCapacityFull(): Checks if the number of added seats has reached the plane's defined capacity.

-Seat Class

*The Seat class has an association relationship with SeatType (Enum). The Seat class contains a variable named type. This variable uses the SeatType enum structure, which indicates whether the seat is ECONOMY or BUSINESS.

Methods:

Set and Get Methods: Organizes properties such as seat number, type, and price, and allows them to be called from other classes.

isReserved() / setReserved(): Manages the occupancy status (boolean) of the seat. This value is marked as true when a reservation is made, and false when cancelled or empty.

-Route Class

*An object of the Route class is contained within the Flight class (Has-A relationship). Internally, it holds the departure and arrival airport (String) and the distance (double).

Methods:

Set and Get Methods: Organizes properties such as departure place, arrival place, and distance, and allows them to be called from other classes.

getRouteInfo(): Returns a simplified format of the route (" (IST) -> (ESB)") to be used in the interface (GUI).

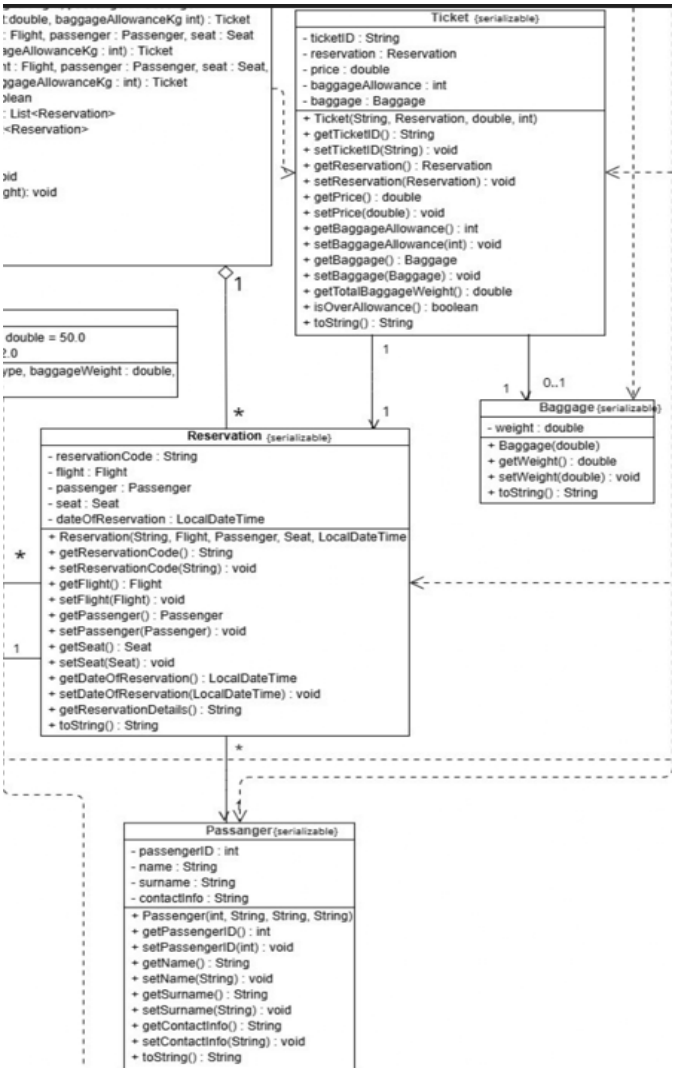
-SeatType Enumeration

*The SeatType enumeration is found as an object within the Seat class.

Properties:

Constants: Defines two fundamental constant values, ECONOMY and BUSINESS. It ensures type safety by preventing manual text entry within the code. (Business class seats were displayed in color in the GUI).

Reservation and Ticketing Module



-Ticket Class

*The Ticket class has an association relationship with the Reservation and Baggage classes.
*A reservation object is contained within the Ticket class. This indicates which reservation the ticket belongs to.
*A baggage object is contained within the Ticket class. This indicates whether the passenger has baggage. Methods:
cancelTicket(): Cancels the ticket by setting the isCancelled status to true.
isOverAllowance(): Checks if the passenger's baggage weight exceeds the baggage allowance on the ticket.

-Baggage Class

*An object of the Baggage class is held within the Ticket class. The Baggage class only holds the weight of the baggage.
Methods:
Setter and getter: Sets the baggage weight and allows it to be retrieved

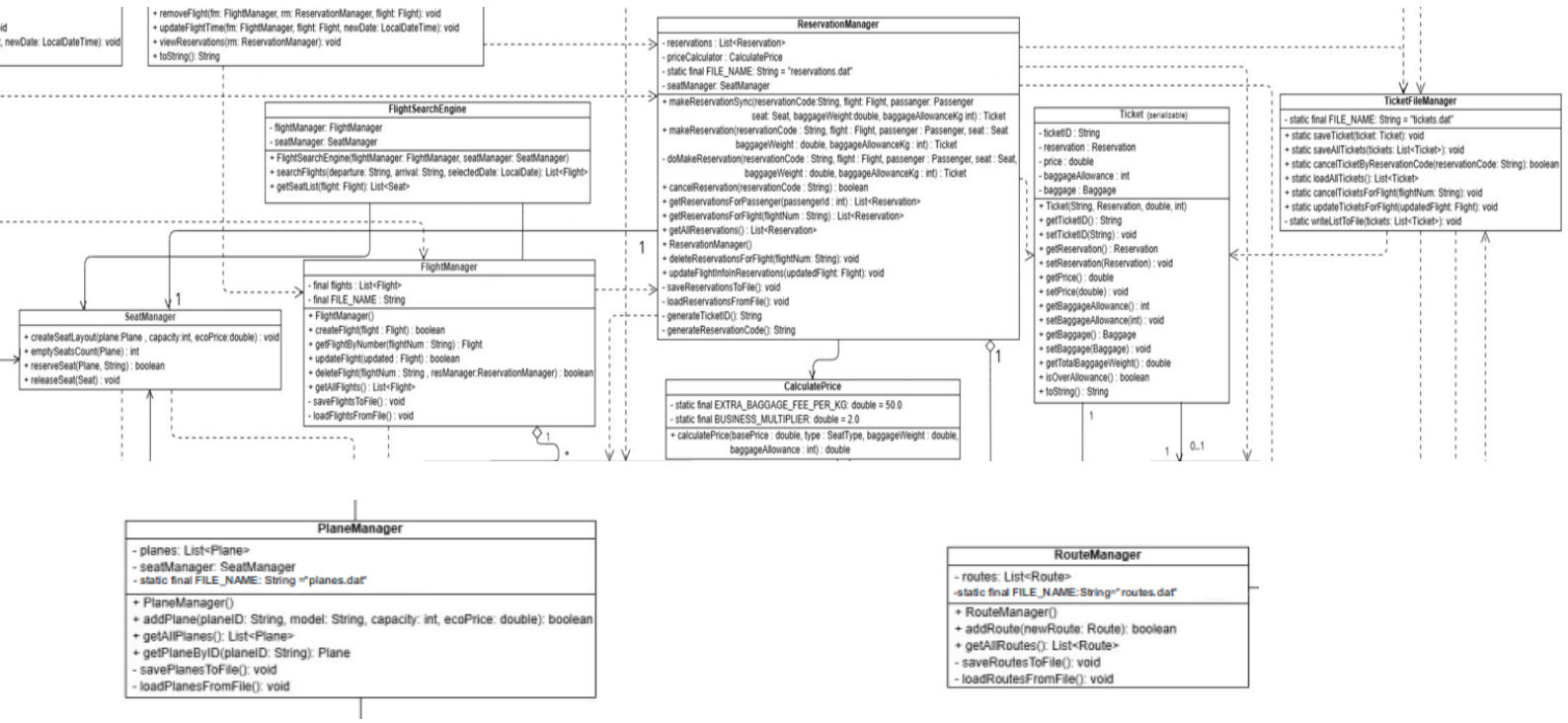
-Reservation Class

*The Reservation class has an association relationship with the Flight, Passenger, and Seat classes.
Methods:
getReservationDetails(): Returns a single summary text (String) by combining the reservation code, flight number, passenger name, and seat number.

-Passenger Class

The Passenger class is used by the Reservation class (Association).
Methods:
Setter and getter: Organizes properties by performing necessary checks and allows them to be called from other classes.

Services and Managers Module



-Flight Manager Class

*The FlightManager class has an Aggregation relationship with the Flight class as it manages the list of Flight objects.

*It also has a Dependency relationship with the TicketFileManager and ReservationManager classes.

Methods:

createFlight(Flight flight): Creates a new flight. It prevents duplicates by checking if the same flight number has been added before.
deleteFlight(String flightNum, ReservationManager resManager): Deletes the flight and triggers the deletion of associated tickets/reservations.
updateFlight(Flight updated): Updates the information of an existing flight and saves the change to the file.

saveFlightsToFile(): Serializes the flights list as is and saves it to the file.
loadFlightsFromFile(): Reads the file when the program starts and restores the list.

-Reservation Manager Class

*The ReservationManager class has an Aggregation relationship with the Reservation class.
*It has Association relationships with the SeatManager and CalculatePrice classes.

Methods:
makeReservation(...): Creates a secure reservation in a synchronized manner, reserves the seat, and generates the ticket.

cancelReservation(String reservationCode): Cancels the reservation and releases the seat.

updateFlightInfoInReservations(Flight updatedFlight): Updates future reservations when flight details change.

saveReservationsToFile(): Saves the current reservation list to the reservations.dat file as serializable. It is automatically called after new reservation, cancellation, or update operations.

loadReservationsFromFile(): Loads registered reservations into memory.

-Plane Manager Class

*The PlaneManager class has an Aggregation relationship with the Plane class.
*It has a Association relationship with the SeatManager class.

Methods:
addPlane(...): Adds a new plane and creates the seat layout using the seatManager object.

getPlaneByID(String planeID): Finds and returns the plane object by ID.

savePlanesToFile(): Saves the plane inventory to the planes.dat file as serializable.

loadPlanesFromFile(): Restores planes and their seat layouts from the file.

-Seat Manager Class

*The SeatManager class has an Dependency relationship with the Seat class.

*The SeatManager class has an Dependency relationship with the Plane class.

Methods:

createSeatLayout(Plane plane, int capacity, double ecoPrice): Creates the seat matrix (Business/Economy) of the plane. The first 3 rows are set as business and the remaining seats are set as economy.

releaseSeat(...): Changes the reservation status of the seat.

-Ticket File Manager Class

*The TicketFileManager class has a Dependency relationship with the Ticket, Flight, and Reservation classes.

Methods:

saveTicket(Ticket ticket): Saves the ticket to the tickets.dat file as serializable.

loadAllTickets(): Reads all tickets from the file.

cancelTicketsForFlight(String flightNum): Cancels tickets en masse for a cancelled flight.

updateTicketsForFlight(Flight updatedFlight): Synchronizes tickets during a flight update. If the plane changes and the capacity decreases, tickets falling outside the capacity are cancelled, while the others are transferred to the new plane.

-Flight Search Engine Class

*The FlightSearchEngine class has an Association relationship with the SeatManager class.
*The FlightSearchEngine class has an Association relationship with the FlightManager class.

Methods:

searchFlights(String departure, String arrival, LocalDate selectedDate): Retrieves suitable flights by filtering based on departure and arrival cities and excluding past flights.

-Calculate Price Class

*The CalculatePrice class has a Dependency relationship with the Seat class.

*The CalculatePrice class has a Dependency relationship with the SeatType class.

Methods:

calculatePrice(double basePrice, Seat seat, double baggageWeight, int baggageAllowance): Calculates the dynamic price based on the seat type and baggage weight.

-Route Manager Class

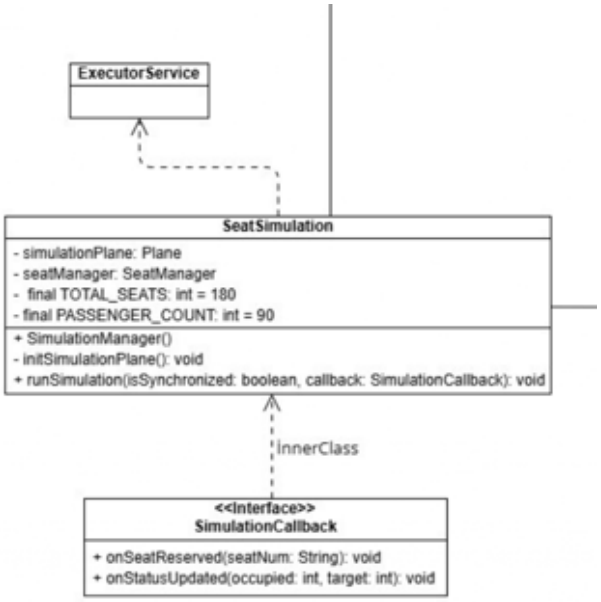
*The RouteManager class has an Aggregation relationship with the Route class.

Methods:

addRoute(Route newRoute): Adds a new route and performs a check to prevent adding duplicates.

saveRoutesToFile(): Writes the defined routes to the routes.dat file as serializable.

loadRoutesFromFile(): Restores the routes from the file.



-Seat Simulation Class

*The SeatSimulation class has an Association relationship with the SeatManager class to create a test environment.

*The SeatSimulation class has an Association relationship with the Plane class to create a test environment.

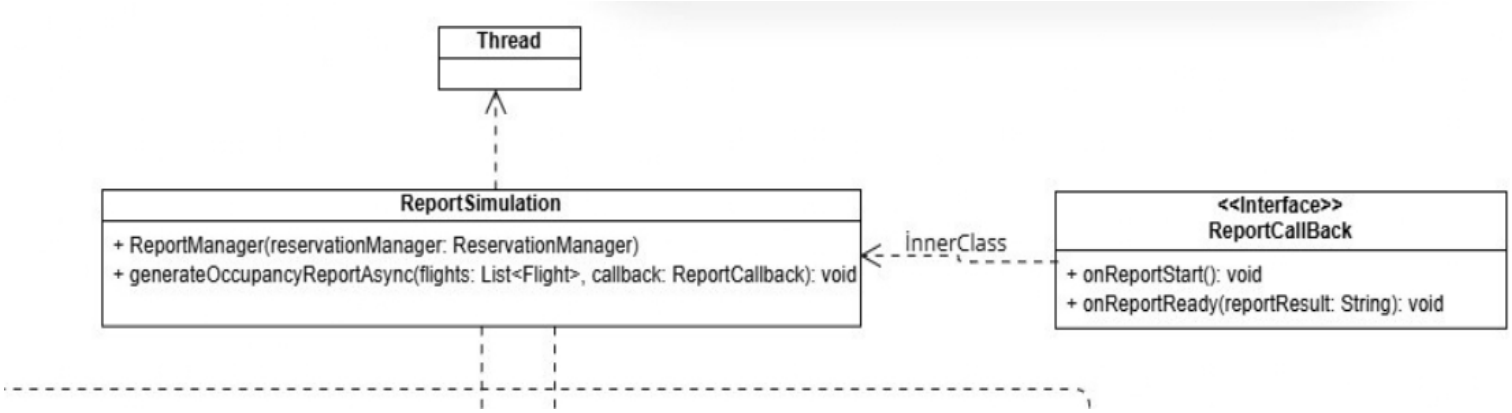
Methods:

runSimulation(boolean isSynchronized, SimulationCallback callback): Starts the simulation.

Synchronized Mod: Uses a synchronized (simulationPlane) block to prevent another process from intervening before one is finished (Thread-Safety) and prevents conflicts.

Non-Synchronized Mod: Intentionally does not place protection and creates a "Race Condition" by adding Thread.sleep(10). In this mode, the error of selling the same seat to multiple people is observed.

initSimulationPlane(): Creates a temporary, empty plane and seat layout named "SIM-TEMP" before each test.



-Report Simulation Class

*The ReportSimulation class has a Dependency relationship with the Thread class for asynchronous reporting.

*It has a Dependency relationship with the TicketFileManager, Flight, and Plane classes to process report data.

*It also hosts the ReportCallback inner interface.

Methods:

generateOccupancyReportAsync(List<Flight> flights, ReportCallback callback): Starts a new Thread to calculate flight occupancy rates without blocking (freezing) the GUI screen and returns the result via callback.